

Tiempo CONUS — Reporte técnico completo

Proyecto: tiempo_CONUS — *Vistazo al Tiempo EEUU* · Canal Meteo TV

Tipo: segmento de vídeo de TV (broadcast 1920×1080, 30 fps) generado por código con [Remotion](#).

Estado: terminado (v1) a 2026-06-25.

Propósito de este documento: dejar por escrito *cómo se hace todo el proceso* —origen de las gráficas, datos, cómo se dibuja el mapa, render y publicación— con el detalle suficiente para que cualquiera (incluida otra sesión de Claude) pueda **reconstruir el producto desde cero** si se pierde.

Todo el código, comentarios y nombres del equipo están en español. Mantener ese idioma al editar.

0. Resumen ejecutivo (TL;DR)

tiempo_CONUS es una composición de Remotion (ConusSegment) que, **en tiempo de render**, descarga del CDN una colección de feeds meteorológicos de EEUU (satélite, radar, temperatura, precipitación, alertas, SPC, sequía, reportes, servicios, terremotos...), construye dinámicamente un **plan de escenas** según qué datos haya disponibles, y dibuja cada escena sobre un **mapa Mapbox con estética "sistema de TV"** (tierra gris con relieve, océano con batimetría). El resultado es un MP4 de ~2 minutos que se renderiza en la máquina GPU (**calamarsa**) cada hora vía cron y se sube al CDN y a Dropbox para playout.

- **Dos máquinas, una sola vía de comunicación (el CDN):**
- **nimbus-01** (`dev.canalmeteo.tv`): corre los *pipelines de datos* (Python) que generan los feeds y los publican al CDN.
- **calamarsa:** corre el *render* (Remotion + Vulkan) y publica el MP4.
- **No comparten disco ni base de datos.** Se comunican **solo a través del CDN** (DigitalOcean Spaces). El frontend nunca habla con nimbus directamente; lee URLs del CDN.
- **CDN:** `https://canalmeteo-public.sfo3.digitaloceanspaces.com` (constante `CDN` en `src/lib/cdn.ts`).
- **Patrón de datos clave:** los feeds traen **valores/geometría**; el frontend **cura coordenadas y nombres en español**. Para datos puntuales por ciudad, un **catálogo en el frontend decide qué se dibuja y cómo**; los ids del feed que no estén en el catálogo se descartan.

1. Topología de infraestructura



- **nimbus-01:** se accede por `ssh root@dev.canalmeteo.tv`. Aloja `/opt/canalmeteo/scripts/*.py` (los pipelines reales y vigentes). **Las copias de estos scripts que pueda haber en calamarsa pueden estar OBSOLETAS** — la fuente de verdad de los pipelines es nimbus. No editar/"sincronizar" esas copias a ciegas; pedir la versión de producción.
- **calamarsa:** máquina de render. Aloja este proyecto Remotion en `/opt/canalmeteo/remotion/tiempo_CONUS`. Tiene GPU (render con `--gl=vulkan`). El usuario `canalmeteo` debe estar en los grupos `render`, `video` para tener acceso a `/dev/dri` (si no, el render cae a CPU/llvmpipe y va lentísimo).
- **CDN:** bucket público `canalmeteo-public` en DigitalOcean Spaces (región sfo3). Subidas con `aws s3 cp --profile do-tor1 --endpoint-url https://sfo3.digitaloceanspaces.com`.

2. Pila tecnológica

Pieza	Versión / detalle
Remotion	4.0.452 (@remotion/cli, @remotion/google-fonts, @remotion/transitions)
React	18.3.1
Mapbox GL JS	3.9.0 (render <i>headless</i> con <code>preserveDrawingBuffer:true</code>)
TypeScript	5.5.4
zod	3.23.8
Fuente	Outfit (Google Fonts, vía @remotion/google-fonts/Outfit) + JetBrains Mono (cifras)
Render	<code>--gl=vulkan</code> en calamarsa; <code>--gl=swangle</code> como <i>fallback</i> software

Pieza	Versión / detalle
Token	
Mapbox	en <code>src/lib/cdn.ts</code> (<code>MAPBOX_TOKEN</code>), estilo base
x	<code>mapbox://styles/mapbox/dark-v11</code>

```
package.json scripts: dev = remotion studio; render = remotion render ConusSegment
out/Tiempo_CONUS.mp4.
```

3. Estructura del proyecto



```
|  └─ TIEMPO CONUS REPORTE.pdf ─ version PDF
|
|
|
|
|  └─ src
|
|
|
|  └─ index.ts ─ registerRoot(Root
|
|
|
|  └─ root.tsx ─ composition - Conussegment
|  calculatenMetadata + todos los Still (mockups)
|
|
|
|  └─ Conussegment.tsx ─ arma la <Series> de escenas según el plan
|  música de fondo
|
|
|
|  └─ types.ts ─ todos los tipos (SalView, AlertsData,
|  SocOutlook, Queke, TmaxPor
|
|
|
|  └─ global.d.ts
|
|
|
|  └─ lib
|
|
|
|  └─ con.ts ─ * CAPA DE DATOS: todas las URLs del CON
|  ranchers, catálogos, plan de escena
|
|
|
|  └─ basemap.ts ─ * dibujo del mapa "sistema" (relieve
|  mapa de fondo, fondo de mapa
|
|
|  └─ las-otras.ts ─ todas las otras cosas de la
|  por importancia
|
|
|
|
|  └─ ...
|  ciclo/viento en español
|
|
|
|  └─ theme.ts ─ sistema de diseño (brano, palette por modo,
|  categorías de alerta
```

└─ mockups.tsx	- catálogos de variantes de base para los
└─ style de comparación	
└─ components	
└─ SatMap.tsx	- * componente Mapbox-en-Remotion (cámara
└─ Overlay.tsx	- TopicBar (barra de título de escena)
└─ Outro.tsx	- cierre ("El tiempo 24/7 en Español") +
└─ cues	
└─ geocolortopus.tsx	- satélite IR CONUS + ciudades (bucle)
└─ RadarLoop.tsx	- radar MRMS (bucle)
└─ Alerts.tsx	- vigilancias/avisos NWS (polígonos + cajas)
└─ CondicionesNow.tsx	- temperatura NBM "ahora" + cajas de ciudad
└─ RadarLoop.tsx	- radar MRMS (bucle)
└─ acumulado_24_h (NBM)	



★ = los tres ficheros que hay que entender primero para reconstruir: `cdn.ts` (de dónde salen los datos), `basemap.ts` + `SatMap.tsx` (cómo se dibuja el mapa).

4. Flujo de datos: orígenes → CDN → frontend

Todo el cableado de datos vive en `src/lib/cdn.ts`. Cada producto tiene un `fetch*` que descarga su manifest/JSON del CDN (con `?ts=${Date.now()}` para saltarse cachés) y lo normaliza a un tipo de `types.ts`. `Root.tsx` → `computeMeta()` lanza **todos** los fetch en paralelo (`Promise.all`) dentro de `calculateMetadata`, y con los resultados arma el plan de escenas y las props de la composición.

4.1. Tabla maestra de feeds

Producto (escena)	Fuente upstream	Pipeline nimbus (aprox.)	Path en el CDN
Satélite IR CONUS	NOAA	<code>goes_ir_pipeline.py</code>	<code>data/goes_ir/conus/manifest.js</code>

Producto (escena)	Fuente upstream	Pipeline nimbus (aprox.)	Path en el CDN
	band a 13		on + PNG RGBA
Satélite GEO COLOR full-disk	NOAA GOES geocolor	satellite_pipeline.py	data/satellite-fd/{view}/manifest.json
Radar	MRMS (reflectividad)	mrms_pipeline.py / radar_compose_pipeline.py	data/mrms/ manifest.json + PNG RGBA
Radar (legacy compuesto)	MRMS	—	data/radar_conus/ manifest.json
Temperatura "ahorita"	NBM (temp 2m)	nbm_temp_pipeline.py (cron :25)	data/nbm/temp/conus/ manifest.json + PNG RGBA
"Radar a futuro"	HRRR REFC (refl.	hrrr_radar_pipeline.py	data/hrrr/radar_fcst/conus/ manifest.json

Producto (escena)	Fuente upstream	Pipeline nimbus (aprox.)	Path en el CDN
	simulada)		
Precip acumulada 24 h	NBM (precip por tipo)	<code>nbm_precip_pipeline.py</code> (cron :40, c/3h)	<code>data/nbm/ precip_accum/ conus/ manifest.json</code>
Condiciones por ciudad	weather-api (METAR/ NBM)	<code>metar_pipeline.py</code> / <code>weather-api.php</code>	<code>data/ cities/ weather.json</code>
Aeropuertos (demoras)	FAA	(FAA pipeline en nimbus)	<code>data/ airports/ delays.json</code>
Índice UV	EPA	(EPA pipeline)	<code>data/uv/ cities.json</code>
Calidad del aire (AQI)	AirNow	<code>sync-airquality.php</code>	<code>data/aqi/ cities.json</code>
Riesgo severo SPC	NOAA SPC outlook día 1	<code>records_spc.py</code>	<code>data/spc/ outlook_day 1.json</code>

Producto (escena)	Fuente upstream	Pipeline nimbus (aprox.)	Path en el CDN
Máxima hoy/ mañana (ráster)	NBM	<code>nbm_temp_pipeline.py (tmax)</code>	<code>data/nbm/tmax/{today,tomorrow}/manifest.json</code>
Varia- ción 24 h (ráster)	NBM (mañana-hoy)	<code>nbm_temp_pipeline.py (tdelta)</code>	<code>data/nbm/tdelta/tomorrow/manifest.json</code>
Máxima por ciudad + población	NBM	<code>nbm_temp_pipeline.py</code>	<code>data/tmax/cities.json</code>
Frentes / centros de presión	análisis de superficie	<code>fetch_ndfd.py</code> (borrasca) o equivalente	<code>data/fronts/fronts.json</code>
Reportes de tormenta 24 h	SPC storm reports	<code>records_spc.py</code> (o reports)	<code>data/reports/storm_reports.json</code>
Sequ	US	<code>sync-drought.php</code> /	<code>data/</code>

Producto (escena)	Fuente upstream	Pipeline nimbus (aprox.)	Path en el CDN
Ía USD M	Drought Monitor	<code>records_daily_pipeline.py</code>	<code>drought/usdm_current.json</code>
Alertas/vigilancias	NWS	<code>alertas_nws.py</code>	<code>data/alertas/alertas_nws.json</code>
Terremoto (última hora)	USGS	(pipeline quake PENDIENTE en nimbus)	<code>data/quake/latest.json</code>

La columna "Pipeline nimbus (aprox.)" es orientativa: la asignación exacta script → feed vive en nimbus-01 y puede haber cambiado. **El contrato fiable es el path del CDN + la forma del dato** (columnas 4 y 6), que es lo que el frontend consume y valida.

4.2. Mecanismos comunes de los fetchers

- **Anti-caché:** todas las URLs llevan `?ts=${Date.now()}`.
- `filterExistingFrames`: los manifests de rásteres animados (IR, radar, precip) pueden ir **por delante** del CDN; se hace `HEAD` a cada frame y se descartan los 404 (conservando el frame si el error es de red transitorio).
- `decimateFrames`: límite de **10 frames** (`MAX_SAT_FRAMES/MAX_RADAR_FRAMES`) para no agotar la GPU al drapear muchas capas raster. Reparte uniformemente y garantiza el frame más reciente.
- **Degradación elegante:** si un feed falla, su `fetch*` devuelve vacío/`null`; la escena correspondiente **no se incluye** en el plan (ver §6). Nunca rompe el render.

5. Cómo se dibuja el mapa

El "look sistema de TV" se eligió en mockups y está en producción. Dos ficheros: `basemap.ts` (las capas) y `SatMap.tsx` (el componente que monta Mapbox en Remotion y drapea datos).

5.1. Base cartográfica `applyBaseMap(map)` (`basemap.ts`)

Sobre el estilo `mapbox://styles/mapbox/dark-v11` ya cargado, en el evento `load`:

1. `setWaterColor` → océano azul base (`BASEMAP.ocean = #2e6088`); recolorea las capas `water` (menos `waterway`).
2. `setLandColor` → tierra gris (`BASEMAP.land = #c6c9cb`); recolorea `background + land/landuse/landcover`.
3. `addReliefRaster` → drapea `relief_conus.png` (raster de relieve sombreado Natural Earth) **bajo** la capa de agua, con `coordinates = RELIEF_RASTER_BOUNDS` (`{west:-142, north:60, east:-52, south:2}`). **Estos bounds DEBEN coincidir** con el `-projwin/-clipsrc` usado al generar el PNG. Da textura de montañas imposible con hillshade vectorial a escala continental.
4. `addBathymetry` → colorea el océano por profundidad con `bathymetry.geojson` (rampa `BATHY_RAMP`: azul medio en plataformas → azul casi negro en fondo). Features ordenadas de somero a profundo (el profundo se pinta encima).
5. `addCoastBorder` → línea de costa nítida (capa `water` de `composite`).
6. `raiseBorders` → sube y realza fronteras admin (países gruesas, estados finas).
7. `hideAutoLabels` → **oculta TODOS los rótulos** de Mapbox (estados, países, carreteras) para un mapa limpio de TV.

Opcional `showPlaceLabels(map)` (prop `placeLabels`): reactiva **solo** los rótulos nativos de *poblaciones menores* (`settlement-minor-label|place-town`) en blanco con halo, para mapas con zoom local (p. ej. el epicentro de un terremoto) donde los marcadores fijos de CONUS no llegan. Las ciudades grandes se ponen con `MAJOR_CITIES` (ver §5.4) para no duplicar.

Gotcha importante: `relief_conus.png` solo cubre CONUS. Fuera de ese bbox (Alaska/Hawái/Caribe) la base sale plana sin relieve.

5.2. El componente `SatMap` (`components/SatMap.tsx`)

Monta un `mapboxgl.Map` dentro de un `AbsoluteFill` de `Remotion`. Claves del render *headless*:

- `preserveDrawingBuffer: true` — OBLIGATORIO para que `Remotion` capture el canvas WebGL.
- `interactive:false, fadeDuration:0, projection:"mercator"`.
- `delayRender/continueRender` — bloquea el frame hasta que el mapa emite `idle` (con timeout de seguridad de 90 s en init, 60 s por frame de satélite). Sin esto, `Remotion` captura el mapa a medio cargar.
- `applyCamera` — hace `map.resize()` **antes** de `cameraForBounds/jumpTo`. El `resize()` es necesario porque el contenedor puede ir dentro de un elemento con `transform` (p. ej. el *push-in* de la portada): sin él, la cámara calcula con tamaño erróneo y deja "el mundo entero" en vez de encuadrar. Usa `cameraForBounds + jumpTo` (más robusto

que `fitBounds` en headless).

Props que acepta (todo se dibuja por encima de la base):

- `sat + showSatellite` → drapea frames raster (image source por `bounds`) y conmuta cuál es visible por frame.
- `polygons` (vigilancias NWS, SPC), `lines` (frentes, con casing y dash), `markers` (centros de presión A/B en HTML), `dots` (reportes de tormenta en un circle layer), `cityMarkers` (rótulos de ciudad).
- `belowBorders` → inserta el raster bajo costas/fronteras (p. ej. precip acumulada, para que los límites se vean encima).
- `animatePolygons` → fundido por frame (escena) vs opacidad fija (Still en frame 0).
- `fitBounds + fitPadding` → encuadre.

5.3. Encuadre único CONUS

Para que **no haya saltos entre escenas**, todas las escenas de mapa nacional comparten el mismo encuadre:

- `CONUS_VIEW = [[-124.7, 24.5], [-66.9, 49.2]]` (48 estados contiguos, sin Caribe/PR).
- `CONUS_PAD = {top:80, bottom:185, left:40, right:40}` — el hueco inferior deja sitio a cajas y barra de tiempo.

5.4. Rótulos de ciudad con descarte por colisión

`cityMarkers` (normalmente `MAJOR_CITIES` de `cities.ts`, **ordenadas por importancia**).

Estilo broadcast: **nombre con halo a la izquierda + punto blanco sobre la coordenada**.

Algoritmo: se recorre por prioridad y solo se añade un rótulo si su caja en pantalla **no se solapa** con una ya colocada (la ciudad más importante gana; la otra se descarta). Esto evita amontonamiento sin curar manualmente cada encuadre.

Para datos puntuales por ciudad (UV, AQI, aeropuertos, tmax, condiciones), existen **catálogos curados separados** en `cdn.ts` (`SERVICE_CITIES`, `AIRPORT_CATALOG`, `TMAX_CITY_CATALOG`, `CONDITION_CITIES`) con `id` → nombre español + lon/lat. La colocación de etiquetas anti-solape de esas escenas usa `placeChips` (en `ServicesMockups.tsx`), con un mecanismo `force` para fijar el lado de etiquetas concretas (p. ej. "Los Ángeles" a la izquierda para dejar sitio a Las Vegas).

6. Plan de escenas (orden y condicionalidad)

Definido en `cdn.ts`: `SCENE_SECONDS` (duración de cada escena), `SceneAvail` (qué hay disponible) y `buildScenePlan(avail)` (construye el orden). `Root.tsx` calcula `avail` a partir de

los fetch y llama a buildScenePlan.

Orden actual (las escenas marcadas *condicional* solo entran si su feed trae datos):

1. quake_intro (3 s) + quake (8 s) — *condicional*: solo si hay terremoto **M≥5.5 en CONUS** (abre el vídeo, antes de la portada).
2. open (3.5 s) — portada.
3. geocolor (8 s) — satélite IR + ciudades.
4. radar (8 s).
5. alerts (8 s) — vigilancias/avisos.
6. spc (8 s) — *condicional* — riesgo severo (aviso).
7. reports (8 s) — *condicional* — reportes de tormenta 24 h (lo que pasó).
8. condiciones (8 s) — temperatura ahora + cajas de ciudad.
9. precip_fcst (8 s) — radar a futuro.
10. precip_accum (8 s) — precip acumulada 24 h.
11. aeropuertos (8 s).
12. uv (8 s).
13. aqi (8 s).
14. drought (8 s) — *condicional* — sequía (antes del bloque de temperatura).
15. tmax_today (8 s) — *condicional*.
16. tvar (8 s) — *condicional* (necesita hoy y mañana).
17. tmax_tomorrow (8 s) — *condicional*.
18. outro (4 s) — cierre.

Nota: la escena fronts (mapa de superficie) está **retirada del plan** (gráfica descartada), pero el feed, el fetcher y el Still Mockup-frentes se mantienen por si se reactiva.

Duración total = suma de escenas (con **cortes secos**, sin solape — ver §7). Con todos los productos presentes ≈ **127.5 s**.

7. Animación y transiciones

- **Bucle a velocidad fija (loopFrameIndex)**: las animaciones (IR, radar, precip) **no** se estiran por toda la escena; se reproducen en bucle a `LOOP_SECONDS = 2.8 s` por vuelta. Con escenas de 8 s, el loop da ~3 vueltas. Se muestra **un solo frame a la vez** (anti-vibración) y el más reciente queda al final de cada vuelta.
 - **Cortes secos (sin cross-dissolve)**: `TRANSITION_FRAMES = 0`. **MOTIVO**: con disolvenca se montan dos mapas WebGL a la vez y el segundo pierde el contexto GL (mapa en negro, p. ej. el radar). Como todas las escenas comparten proyección y base idéntica, el corte es **imperceptible** (el fondo no salta; cada escena hace su propio fade-in del contenido). **No reintroducir cross-dissolve**.
-

8. Sistema de diseño (`theme.ts`, `overlay.tsx`)

- **BRAND** — paleta Canal Meteo (navy, azul, naranja, blanco) + **Modo Rojo** (`alertRed #d81e3f`, deliberadamente distinto del rojo ladrillo de *Trópico*).
 - **palette(mode)** — devuelve colores según `mode`: "normal" (navy/azul) o "alert" (crimson). El modo lo calcula `computeMode(alerts)`: **Modo Rojo si hay tornado o tiempo severo** en los conteos del feed de alertas. Se puede forzar con `forceMode`.
 - **ALERT_CATEGORIES** — mapa tipo `→ {label, color, icono}` para las cajas de vigilancias (tornado, tiempo severo, inundación, calor, invierno, costera, viento, fuego).
 - **TopicBar** (`Overlay.tsx`) — barra de título de cada escena, coloreada por `palette().topicColor`.
 - **Outro.tsx** — cierre: logo + "El tiempo 24/7 en Español" + redes `@canalmeteotv / @alberteltiempo`.
-

9. Música de fondo

- Fichero: `public/audio/Musica_Costa_a_Costa.mp3` (208 s).
 - Cableada en `ConusSegment.tsx` con `<Audio`
`src={staticFile("audio/Musica_Costa_a_Costa.mp3")}` `loop` `volume={fn}` `>`
como hermana de la `<Series>`.
 - **loop** (por si el vídeo crece) y **volumen bajo (0.22)** con **fundido de entrada** (1 s) y **de salida** (1.5 s antes del final), calculado sobre `totalFrames` = suma de `escenas del plan`.
 - Verificado: el MP4 sale con pista de audio AAC además del vídeo H.264.
-

10. Render y publicación (`render.sh` + `cron`)

`tiempo_CONUS/render.sh` (corre en calamarsa):

1. Renderiza `ConusSegment` con `remotion` `render src/index.ts ConusSegment`
`<out>.mp4 --gl=vulkan --concurrency=4 --scale=1`.
2. **concurrency=4** (no 8): las escenas drapean raster pesado (relieve + IR/radar); con 8 pestañas WebGL a la vez la GPU se quedaba sin memoria ("Failed to initialize WebGL").
3. Escribe `render-status.json` con progreso (parseando "Rendered/Encoded N/M").
4. Valida el MP4 (existe, >1 MB, duración >5 s con `ffprobe`).
5. Copia a `tiempo-conus-latest.mp4`.
6. **Sube al CDN** (`data/videos/tiempo-conus/...mp4 + ...-latest.mp4 + logs`) y **a Dropbox** (`/Payout/RenderFarm/TIEMPO_CONUS.mp4`).
7. Escribe `render-status.json/render-history.json` y los sube.

8. Retiene los últimos 24 MP4 locales.

Flags: `--no-upload` (solo local), `--concurrency=N`, `--gl=swangle` (máquina sin GPU), `--triggered-by=etiqueta`.

Cron (en calamarsa):

```
#!/bin/bash
/opt/canalmeteo/remotion/render.sh --triggered-by=etiqueta
/opt/canalmeteo/remotion/render.sh --triggered-by=etiqueta
```

- **A :33** cada hora: separado del render del *Trópico* (huracanes, a :05) y de los chequeos NHC `*/5`, para **no competir por la única GPU**.
- **flock** evita que un render se solape consigo mismo (cada render ~4–5 min).
- ⚠ El cron **publica automáticamente** (CDN + Dropbox) cada hora. Para que renderice sin subir, añadir `--no-upload`.

El render del Trópico (proyecto hermano huracanes) usa `/opt/canalmeteo/remotion/render.sh` y lo dispara `nhc_check_and_render.py` (cron `:05 + */5` solo-si-cambió). Mismo formato de `status/history`.

11. Mockups / Stills (Remotion Studio)

`Root.tsx` registra, además de la `Composition ConusSegment`, muchos **Still** para pulir el look y exportar PNG sin renderizar el vídeo entero. Algunos llevan su propio `calculateMetadata` que descarga datos reales:

- `Mockup-condiciones`, `Mockup-aeropuertos`, `Mockup-uv`, `Mockup-aqi`.
- `Mockup-frentes`, `Mockup-reportes`, `Mockup-sequia` (datos reales, `animate:false`).
- `Mockup-quake-intro`, `Mockup-quake`.
- `Mockup-spc`, `Mockup-tmax-hoy`, `Mockup-tvar-manana`, `Mockup-tmax-manana`.
- `Mockup-<variante>` de base cartográfica (de `MOCKUPS/RELIEF MOCKUPS/SYSTEM MOCKUPS`).

Render de un Still: `npx remotion still src/index.ts <id> /tmp/x.png --gl=vulkan`.

Render de un frame de la composición: `npx remotion still src/index.ts ConusSegment /tmp/x.png --gl=vulkan --frame=N`.

Detalle a recordar: en un `Still`, `durationInFrames=1`, así que las animaciones por frame (`springs`, `fadeOut` con ventana `[dur-10, dur]`) salen mal en el frame 0. Por eso varios componentes aceptan `animate={false}` (`springs=1`, sin `fadeOut`) y los mockups lo usan.

12. Flags de test y trabajo PENDIENTE/bloqueado

Dos flags en `cdn.ts` están en `true` para poder ver escenas cuyo feed real aún no publica nimbus. Hay que ponerlos a `false` en producción limpia cuando los pipelines estén desplegados:

- `QUAKE_TEST = true` — si el CDN no tiene `data/quake/latest.json`, consulta USGS `4.5_day.geojson` aplicando el **filtro de producción** ($M \geq 5.5$ + bbox CONUS lon $-125...-66$, lat $24...50$ + últimas 6 h). Así solo abre el vídeo un sismo realmente cualificado.
- `TMAX_TEST = true` — rellena con valores de muestra (`SAMPLE_TMAX`) las ciudades del catálogo de máximas que el feed aún no trae (Cleveland, Raleigh, New Orleans, Boise, Washington) y simula la máxima de HOY (mañana -2 °F) en runs de tarde.

Pipelines pendientes en nimbus-01 (BLOQUEAN producción 100% real):

1. **Terremoto** → publicar `data/quake/latest.json` (origen USGS). Contrato:

```
```.json

{ "updated": <unix>,

"quake": { "id": "us7000...", "mag": 6.4, "place": "18 km al SE de Ridgecrest, California",

"lon": -117.5, "lat": 35.6, "depthKm": 9, "time": <unix>,

"timeLabel": "Hoy 09:41 hora local", "tsunami": 0, "felt": 24300 } }

...


```

Sin sismo cualificado → fichero ausente/404 o `{"quake": null}`. El pipeline filtra: **CONUS** (bbox lon  $-125...-66$ , lat  $24...50$ ; excluye Alaska/Hawái/PR),  **$M \geq 5.5$** , **últimas 6 h**, se queda con el más fuerte/reciente. `place/timeLabel` en español. Cron frecuente (2–5 min).

1. **Máxima por ciudad + población** → en `data/tmax/cities.json`, incluir los ids **BOI, MSY, CLE, RDU, DC** en `today/tomorrow`, y el bloque `population.{today,tomorrow}. {heat90,heat100,cold32}` (personas por encima de 90 °F / 100 °F / por debajo de 32 °F). El frontend ya lo dibuja (cajas de población en `tmax` y en `SPC`).

---

## 13. Cómo reconstruirlo desde cero (receta)

---

Si el proyecto se pierde y hay que rehacerlo:

1. **Andamiaje Remotion:** `package.json` con las deps de §2; `src/index.ts` → `registerRoot(Root)`.

2. **Assets de base:** generar `public/relief_conus.png` (relieve sombreado Natural Earth recortado a `RELIEF_RASTER_BOUNDS = {west:-142,north:60,east:-52,south:2}`) y `public/bathymetry.geojson` (bandas de profundidad). **Los bounds del PNG y de RELIEF\_RASTER\_BOUNDS deben coincidir exactamente.**
  3. **Mapa "sistema"** (`basemap.ts` + `SatMap.tsx`): implementar `applyBaseMap` (agua, tierra, relieve bajo agua, batimetría, costas, fronteras, ocultar rótulos) y el componente `Mapbox-en-Remotion` con `preserveDrawingBuffer`, `delayRender/idle`, `applyCamera` con `resize()` previo. Encuadre `CONUS_VIEW/CONUS_PAD`.
  4. **Capa de datos** (`cdn.ts`): para cada producto de la tabla §4.1, escribir el `fetch*` que lee `CDN/<path>?ts=...`, valida y normaliza al tipo de `types.ts`. Reutilizar `filterExistingFrames` y `decimateFrames` para rásteres animados (límite 10 frames). Catálogos curados de ciudad (`SERVICE_CITIES`, `AIRPORT_CATALOG`, `TMAX_CITY_CATALOG`, `CONDITION_CITIES`, `MAJOR_CITIES`).
  5. **Plan de escenas:** `SCENE_SECONDS`, `SceneAvail`, `buildScenePlan` (orden de §6, condicionalidad por disponibilidad de datos).
  6. **Escenas:** una por producto (ver `src/scenes/`), todas sobre `SatMap` con el mismo encuadre; bucle con `loopFrameIndex`; fade-in propio; `TopicBar` con color de `palette(mode)`.
  7. **Composición:** `Root.tsx` con `Composition id="ConusSegment" + calculateMetadata (= computeMeta: Promise.all de todos los fetch → buildScenePlan → props)`. `ConusSegment.tsx` arma la `<Series>` por el plan + música de fondo. **Cortes secos (TRANSITION\_FRAMES = 0).**
  8. **Render/publicación:** `render.sh` (Vulkan, concurrency 4, validación, subida CDN+Dropbox, status/history) + cron a `:33`.
  9. **Producción limpia:** poner `QUAKE_TEST` y `TMAX_TEST` a `false` cuando los pipelines de nimbus publiquen los feeds reales (§12).
- 

## 14. Gotchas / lecciones aprendidas

---

- `map.resize()` antes de `fitBounds/cameraForBounds` o la cámara calcula con tamaño erróneo (mapa "del mundo entero").
- `preserveDrawingBuffer:true` o Remotion captura un canvas vacío.
- **Esperar idle** con `delayRender/continueRender` o el mapa sale a medio cargar.
- **Límite ~10 frames raster y --concurrency=4** o la GPU revienta ("Failed to initialize WebGL").
- **No cross-dissolve** entre escenas de mapa (dos contextos WebGL → el segundo en negro). Cortes secos.
- `relief_conus.png` solo cubre **CONUS** (fuera, base plana).
- **En Still (durationInFrames=1)** las animaciones por frame fallan en frame 0 → usar `animate={false}` en mockups.
- **Render por GPU:** `canalmeteo` debe estar en grupos `render,video` (acceso a

/dev/dri); si no, render por CPU (llvmpipe), lentísimo.

- **Pipelines = nimbus; render = calamarsa; única vía = CDN.** No asumir que las copias de pipelines en calamarsa están al día.
- **Catálogo manda:** un id de ciudad que no esté en el catálogo del frontend **no se dibuja**, aunque el feed lo traiga (y al revés: el frontend no inventa datos, solo cura coords/nombres).

---

## 15. Referencias rápidas

---

- CDN base: <https://canalmeteo-public.sfo3.digitaloceanspaces.com>
- SSH nimbus (pipelines): `ssh root@dev.canalmeteo.tv`
- Repo git: `/opt/canalmeteo` (monorepo; este proyecto en `remotion/tiempo_CONUS`).
- Dropbox playout: `/Playout/RenderFarm/TIEMPO_CONUS.mp4`
- Proyecto hermano: `remotion/huracanes` (*Vistazo al Trópico*), mismo patrón de render/status.

*Documento generado el 2026-06-25. Mantener actualizado al añadir escenas, cambiar el plan o desplegar pipelines.*